

RAPPORT DE STAGE D'ÉTÉ

présenté à

**L'École Nationale d'Électronique et des
Télécommunications de Sfax**

en vue de la validation du stage de

**Diplôme National d'Ingénieur en :
Génie Télécommunication**

par

Mariem AJILI

**Intelligence artificielle et gestion adaptative du risque :
application du critère de Kelly à la navigation autonome
d'un robot mobile**

Réalisé au sein de **AUTRIS SAS** (du 10 juillet au 31 août 2026)

M. Walid BEN AYED — Encadrant de stage

Remerciements

Je tiens à remercier **AUTRIS SAS** pour l'opportunité qui m'a été donnée d'effectuer ce stage à distance sur un sujet à la croisée de la finance quantitative, de l'intelligence artificielle et de la robotique.

Je remercie tout particulièrement mon encadrant, **M. Walid Ben Ayed**, pour son suivi régulier, sa disponibilité et ses conseils techniques tout au long de ces huit semaines, malgré la distance.

Je remercie également le corps enseignant de l'**ENETCOM** pour la formation qui m'a permis d'aborder ce projet dans de bonnes conditions.

Résumé

Ce stage, réalisé au sein d'**AUTRIS SAS**, porte sur l'application du **critère de Kelly** — un principe issu des mathématiques financières utilisé pour le dimensionnement optimal d'une mise en pari ou en investissement — à la navigation autonome d'un robot mobile.

L'objectif est de permettre à un robot d'adapter dynamiquement son comportement (vitesse, distance de sécurité) en fonction d'un niveau de risque estimé à partir de ses données de perception, plutôt que de suivre une politique de navigation figée.

Le travail s'est déroulé en deux phases. La première a consisté à mettre en place une chaîne de navigation autonome classique, combinant cartographie simultanée et localisation (SLAM, via `slam_toolbox`) et planification de trajectoire (Nav2), sous ROS 2 Jazzy et Gazebo Harmonic.

La seconde phase a porté sur l'intégration d'un module de gestion adaptative du risque inspiré du critère de Kelly, modulant en temps réel la vitesse maximale et la distance de sécurité du robot via la reconfiguration dynamique des paramètres de Nav2.

Une comparaison entre la navigation classique et la navigation risque-adaptative a permis d'évaluer l'intérêt de cette approche pour la sécurité de la navigation autonome.

Mots-clés : ROS 2, SLAM, Nav2, navigation autonome, critère de Kelly, gestion du risque, robotique mobile.

Sommaire

1. Introduction générale

2. Présentation de l'entreprise d'accueil — AUTRIS SAS

2.1 Présentation générale

2.2 Domaines d'expertise

2.3 Recherche, développement et innovation

2.4 Projets et réalisations

3. Contexte théorique

3.1 Le critère de Kelly

3.2 Adaptation à la navigation autonome

3.3 SLAM et navigation autonome (Nav2)

4. Environnement et outils

5. Méthodologie et réalisation

5.1 Phase 1 — Navigation SLAM classique avec Nav2

5.1.1 Mise en place de l'environnement de simulation et choix du robot

5.1.2 Architecture de la chaîne de simulation

5.1.3 Difficultés techniques rencontrées et résolues

5.1.4 Résultats de la Phase 1

5.2 Phase 2 — Intégration du critère de Kelly

5.2.1 Choix de modélisation

5.2.2 Implémentation

5.2.3 Adaptation de l'infrastructure existante

6. Résultats et comparaison

6.1 Protocole de comparaison

6.2 Résultats observés

7. Discussion et limites

8. Conclusion et perspectives

9. Bibliographie

10. Annexes

1. Introduction générale

Dans le cadre de ma formation en Génie Télécommunication à l'ENETCOM, j'ai effectué un stage au sein d'AUTRIS SAS, entreprise d'ingénierie spécialisée dans les systèmes industriels, l'automatisation et la robotique intelligente.

Ce stage, d'une durée de huit semaines (10 juillet au 31 août 2026), s'est déroulé en distanciel depuis Sfax, avec des réunions de suivi en présentiel.

Le sujet proposé par mon encadrant, M. Walid Ben Ayed, visait à établir un pont entre un domaine que j'avais initialement exploré — la sécurité financière — et la robotique autonome : appliquer le critère de Kelly, formule de dimensionnement de mise utilisée en finance et en théorie de la décision, à la gestion du risque dans la navigation d'un robot mobile simulé.

L'objectif général du stage était double : d'une part, mettre en place une chaîne de navigation autonome complète et fonctionnelle (cartographie, localisation, planification de trajectoire) ; d'autre part, développer et intégrer un mécanisme d'adaptation du comportement du robot au risque perçu, puis comparer les performances de sécurité entre navigation classique et navigation risque-adaptative.

2. Présentation de l'entreprise d'accueil — AUTRIS SAS

2.1 Présentation générale

AUTRIS SAS est une société française d'ingénierie spécialisée dans la conception, le développement et l'intégration de solutions technologiques destinées aux secteurs industriels. L'entreprise intervient principalement dans les domaines de l'automatisation industrielle, de l'instrumentation, des systèmes de contrôle-commande, de l'informatique industrielle, des systèmes embarqués, de la robotique et de l'intelligence artificielle appliquée aux systèmes industriels et autonomes.

Implantée en France, AUTRIS dispose également d'une filiale en Tunisie, lui permettant de renforcer ses capacités d'ingénierie, de développement et de recherche, tout en favorisant les collaborations avec les acteurs industriels, les établissements d'enseignement supérieur et les structures de recherche.

Grâce à une approche pluridisciplinaire associant automatisme, informatique, instrumentation, électronique, intelligence artificielle et robotique, AUTRIS accompagne ses partenaires dans les différentes phases de leurs projets, depuis l'analyse des besoins et la conception jusqu'au développement, à l'intégration, aux essais, à la mise en service et à la validation des solutions mises en œuvre.

2.2 Domaines d'expertise

Les activités d'AUTRIS s'articulent autour de plusieurs domaines complémentaires de l'ingénierie et des technologies avancées :

- **Automatisation industrielle** : étude et développement de systèmes automatisés, programmation d'automates industriels, intégration de systèmes de supervision et d'interfaces homme-machine, solutions de contrôle, d'acquisition et de traitement des données.
- **Instrumentation et contrôle-commande** : intégration de capteurs, d'actionneurs, de systèmes de mesure et de réseaux de communication industriels.
- **Systèmes embarqués, robotique mobile et intelligence artificielle** : perception de l'environnement, vision par ordinateur, fusion de données multi-capteurs, localisation, cartographie, navigation autonome, planification de trajectoires et prise

de décision intelligente.

2.3 Recherche, développement et innovation

AUTRIS accorde une place importante aux activités de R&D, avec pour objectif d'intégrer les technologies émergentes aux problématiques industrielles et aux systèmes autonomes. Les travaux portent notamment sur l'automatisation avancée, les systèmes intelligents, la robotique mobile autonome, la perception multi-capteurs, la vision par ordinateur, la fusion de données, le SLAM, la navigation intelligente, l'intelligence artificielle embarquée ainsi que les méthodes d'apprentissage automatique, d'apprentissage profond et d'apprentissage par renforcement.

La présence d'AUTRIS en France et en Tunisie contribue au développement de collaborations entre le monde industriel et le milieu académique, et permet de conduire des projets associant ingénierie, recherche appliquée, expérimentation et développement de solutions innovantes.

2.4 Projets et réalisations

AUTRIS intervient sur des projets couvrant aussi bien les applications industrielles que les systèmes intelligents et robotiques : modernisation de systèmes d'acquisition de données, développement et migration de systèmes SCADA, intégration d'automates programmables, développement d'interfaces de supervision, intégration de capteurs et d'actionneurs, projets dans les secteurs de l'énergie, du pétrole et du gaz (automatisation, supervision, pré-commissioning, commissioning), ainsi que des projets de R&D en intelligence artificielle et robotique : navigation autonome des robots mobiles, fusion de données LiDAR-caméra, perception de l'environnement, SLAM, détection et suivi d'obstacles, planification de trajectoires et apprentissage par renforcement appliqué à la navigation et à la prise de décision.

C'est dans ce dernier axe de R&D que s'inscrit le présent stage.

3. Contexte théorique

3.1 Le critère de Kelly

Formulé originellement par John L. Kelly Jr. dans son article séminal de 1956 au sein des *Bell System Technical Journal*, le critère de Kelly vise à déterminer la fraction optimale f^* d'une ressource (typiquement un capital financier) à engager dans une suite d'épreuves à issue incertaine, afin de maximiser le taux de croissance logarithmique espéré sur le long terme tout en minimisant le risque de ruine.

**FORMULE GÉNÉRALE DU CRITÈRE DE
KELLY**

$$f^* = \frac{p}{L} - \frac{q}{G}$$

où p est la probabilité de succès, $q = 1 - p$ la probabilité d'échec, L la fraction de ressource perdue en cas d'échec, et G le facteur de gain en cas de succès.

3.2 Adaptation à la navigation autonome

Dans le cadre de ce travail, nous transposons ce formalisme mathématique à la cinématique et à la sécurité d'un robot mobile autonome en remplaçant la notion de pari monétaire par le dimensionnement dynamique de la vitesse et de l'espace de sécurité :

Concept en Finance Quantitative	Équivalence en Navigation Robotique Adaptative
Capital misé (f^*)	Fraction de la vitesse maximale nominale autorisée au contrôleur local ($v_{x,\max}$)
Probabilité de succès (p)	Probabilité estimée que la trajectoire projetée soit exempte de collision (p_{safe})
Fraction perdue en échec (L)	Coût ou sévérité d'une collision potentielle (arrêt d'urgence, choc mécanique, immobilisation)
Facteur de gain (G)	Bénéfice marginal d'une progression rapide (gain de temps de parcours logistique)

3.3 SLAM et navigation autonome (Nav2)

Le SLAM (*Simultaneous Localization and Mapping*) résout le problème fondamental de la construction conjointe de la carte de l'environnement et de l'estimation de la pose du robot (x, y, θ) . Le package `slam_toolbox` utilisé s'appuie sur la corrélation de scans laser LiDAR 2D et la fermeture de boucle par optimisation de graphe de poses.

La pile Nav2 (Navigation2) structure l'autonomie robotique sous forme de serveurs de cycle de vie ROS 2 :

- **Planner Server** : calcule un chemin global sans collision sur la costmap globale (via NavFn ou Smac Planner).
- **Controller Server** : génère les commandes de vitesse vélocimétriques `cmd_vel` via l'algorithme MPPI (*Model Predictive Path Integral*), optimisant des milliers de trajectoires candidates sous contraintes de dynamique.
- **Costmap 2D** : représente les grilles d'occupation 2D décomposées en couches (statique, obstacles laser, et couche d'inflation).
- **Behavior Tree Navigator** : coordonne la logique de mission, la replanification et les actions de récupération en cas de blocage.

4. Environnement et outils

Pour assurer des développements conformes aux derniers standards industriels tout en garantissant des performances d'exécution fluides sous une machine de développement sous Windows Subsystem for Linux (WSL2), les choix logiciels et matériels suivants ont été adoptés :

Composant Système	Choix retenu	Justification technique
Système d'exploitation	Ubuntu 24.04 LTS (WSL2)	Support natif du kernel Linux avec intégration graphique Wayland (WSLg)
Framework Middleware	ROS 2 Jazzy Jalisco	Dernière version LTS de ROS 2 offrant des performances accrues et un support étendu jusqu'en 2029
Moteur de simulation	Gazebo Harmonic	Successeur moderne d'Ignition Gazebo avec support officiel sous ROS 2 Jazzy
Modèle de Robot	tugbot	Plateforme AMR industrielle réaliste, stable sous Gazebo et optimisée pour WSL2
Module SLAM	slam_toolbox	Cartographie robuste par graphe de poses, gestion du mode localisation synchrone
Pile de navigation	Nav2 (Contrôleur MPPI)	Évaluation stochastique prédictive de trajectoires évitant les blocages locaux
Visualisation 3D	RViz2	Supervision des nuages de points LiDAR, grilles d'occupation, buts 2D et repères TF
Orchestration shell	tmux	Automatisation multi-terminaux du lancement simultané des nœuds ROS 2 et bridges
Langage & API	Python 3 (rclpy)	Développement rapide du package <code>kelly_nav</code> et clients de services ROS 2

5. Méthodologie et réalisation

5.1 Phase 1 — Navigation SLAM classique avec Nav2

5.1.1 Mise en place de l'environnement de simulation et choix du robot

La mise en œuvre de la chaîne de simulation a requis un travail approfondi d'investigation et de dépannage système. Plusieurs plateformes ont été successivement déployées et évaluées avant d'arriver à une solution pérenne :

- **TurtleBot4 (ROS 2 Jazzy)** : testé initialement pendant une semaine, il a dû être écarté en raison d'incompatibilités majeures entre Gazebo Harmonic et les meshes de rendu 3D, ainsi que de ruptures dans l'arbre de transformations TF (repère parent introuvable pour la caméra stéréoscopique).
- **TurtleBot3 Burger** : rapidement opérationnel pour valider la communication entre les topics LiDAR et RViz, il a été jugé trop basique et éloigné des exigences d'un cas d'usage industriel pour le sujet.
- **iRobot Create3** : bien que prometteur, ce modèle a rencontré des blocages sévères sous WSL2 : temps de rendu LiDAR prohibitif sans accélération GPU matérielle directe, provoquant un facteur temps réel (*Real Time Factor* — RTF) chutant à 0.01 (simulation 100 fois plus lente que l'horloge réelle).
- **tugbot (retenu)** : modèle de tracteur mobile industriel de type AMR, particulièrement stable sous Gazebo Harmonic, assurant un RTF proche de 0.95 et sur lequel l'intégralité du travail de cartographie et de régulation a été accomplie.

5.1.2 Architecture de la chaîne de simulation

La chaîne de simulation repose sur les éléments suivants :

- **Gazebo Harmonic**, exécutant le monde simulé (entrepôt), avec le robot tugbot ;
- des **bridges ROS 2 ↔ Gazebo** (`ros_gz_bridge`) exposant les topics `/clock`, `/cmd_vel`, `/tf`, `/scan` et `/odom` côté ROS 2 ;
- un **static transform publisher** reliant le repère `base_link` au repère du LiDAR ;
- **slam_toolbox**, en charge de la construction de la carte à partir du scan LiDAR et de l'odométrie ;

- **Nav2** (`controller_server`, `planner_server`, `bt_navigator`, `behavior_server`, `smoother_server`, costmaps local et global), assurant la planification et le suivi de trajectoire ;
- **RViz2**, pour la visualisation et l'envoi d'objectifs de navigation (« 2D Goal Pose »).

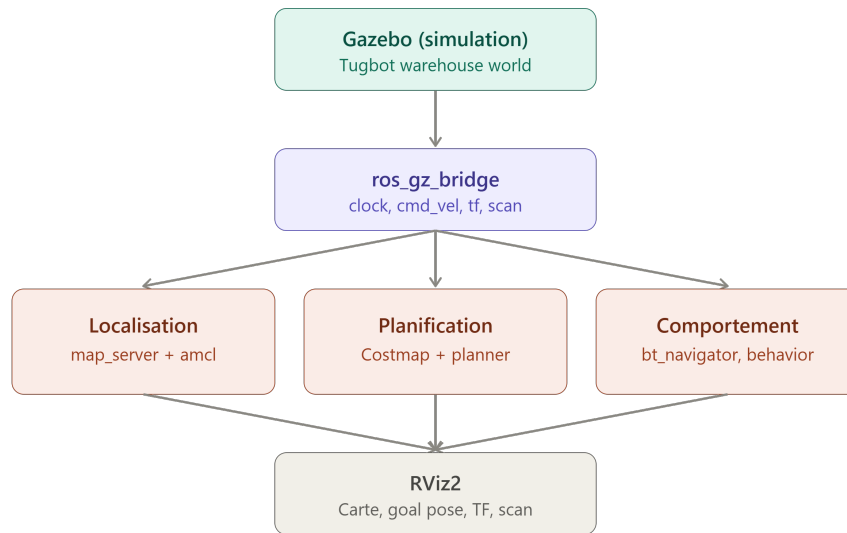


Figure 1 — Chaîne de simulation et de navigation : Gazebo alimente les topics ROS 2 via `ros_gz_bridge` (`clock`, `cmd_vel`, `tf`, `scan`), qui alimentent en parallèle la localisation (`map_server/amcl` ou `slam_toolbox`), la planification (`costmap + planner`) et la gestion du comportement (`bt_navigator`, `behavior server`) ; RViz2 permet la visualisation et l'envoi d'objectifs. En Phase 2, le nœud `kelly_monitor_node` (non représenté ici) s'intercale en lecture sur `/scan` et `/odom`, et agit sur le bloc Planification via reconfiguration dynamique des paramètres (vitesse max, rayon d'inflation).

5.1.3 Difficultés techniques rencontrées et résolues

- **Repère `base_footprint` vs `base_link`** : incohérence de configuration corrigée dans les paramètres Nav2.
- **Configuration MPPI trop lourde pour WSL2** : allégement des paramètres du contrôleur, fréquence de contrôle remontée à environ 20 Hz pour une exécution fluide.
- **Pose initiale AMCL** : le repère `map` n'est publié qu'après réception d'une pose initiale (2D Pose Estimate dans RViz ou message `/initialpose`) — un délai trop important entre le lancement de la pile et l'envoi de cette pose provoque un timeout du gestionnaire de cycle de vie (*lifecycle manager*) de Nav2.

5.1.4 Résultats de la Phase 1

- Carte de l'environnement (entrepôt simulé) construite avec succès et sauvegardée.

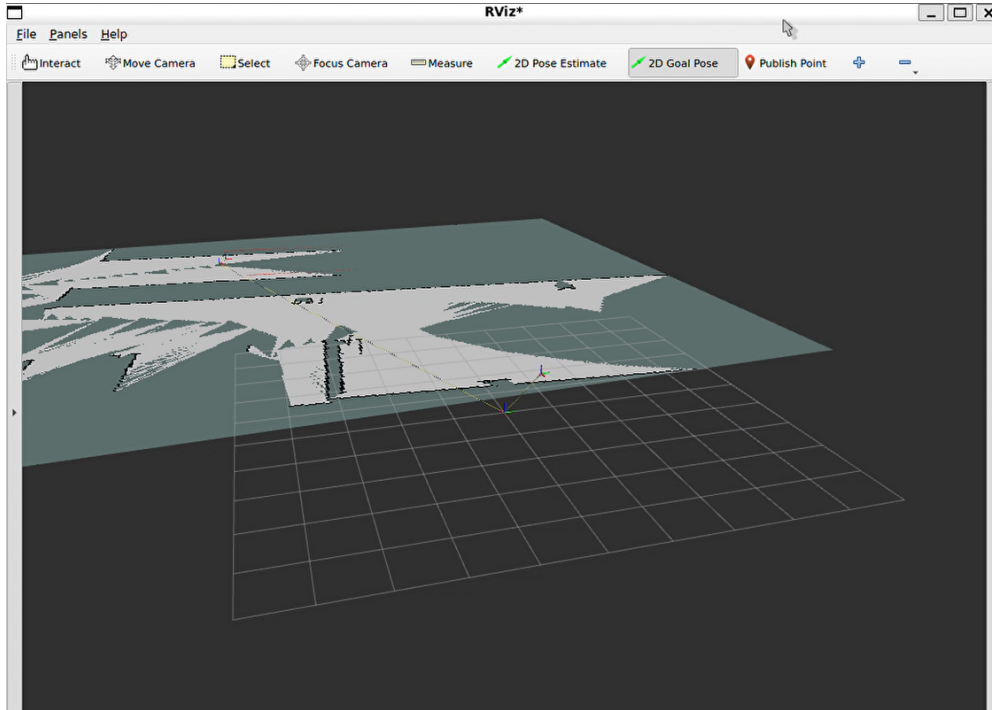


Figure 2 — Carte de l'entrepôt en cours de construction dans RViz2 via `slam_toolbox`, avec la trajectoire parcourue par le robot visible en pointillés.

- Navigation autonome fonctionnelle vers un objectif défini dans RViz, avec planification et suivi de trajectoire corrects.
- Test d'évitement d'obstacles réussi (contournement d'une étagère).
- Vidéo de démonstration enregistrée.

(Liste complète des nœuds et topics détaillée en Annexe B.)

5.2 Phase 2 — Intégration du critère de Kelly

5.2.1 Choix de modélisation

Le critère de Kelly a été adapté à la navigation selon la correspondance suivante :

Finance	Navigation
Capital à miser	Vitesse maximale autorisée
Probabilité de succès p	Probabilité de trajectoire sûre (estimée à partir du LiDAR)
Fraction perdue en cas d'échec L	Sévérité potentielle d'une collision (ex. arrêt d'urgence, perte de temps/dégâts)
Fraction gagnée en cas de succès G	Bénéfice relatif d'une navigation rapide (gain de temps)
Fraction misee f^*	Fraction de la vitesse nominale maximale utilisée

La probabilité p est calculée à partir de deux indicateurs de risque, chacun interpolé linéairement entre un seuil critique ($p = 0$) et un seuil jugé sûr ($p = 1$), le minimum des deux étant retenu :

- la **distance minimale** aux obstacles détectés par le LiDAR dans un cône avant (120° par défaut) ;
- le **temps avant collision estimé** (TTC), calculé comme le rapport entre cette distance minimale et la vitesse courante du robot.

La fraction de Kelly obtenue, $f^* = p/L - q/G$ (avec $q = 1-p$), module ensuite en temps réel :

- la **vitesse maximale** transmise au contrôleur MPPI de Nav2 (paramètre `FollowPath.vx_max`) ;
- le **rayon d'inflation** du costmap local (paramètre `inflation_layer.inflation_radius`), c'est-à-dire la distance de sécurité maintenue autour des obstacles.

5.2.2 Implémentation

Un package ROS 2 (`kelly_nav`) a été développé, contenant un nœud (`kelly_monitor_node`) qui :

- s'abonne aux topics `/scan` et `/odom` pour estimer le risque en continu ;
- publie des topics de diagnostic (`/kelly/p_safe`, `/kelly/fraction`, `/kelly/ttc`, `/kelly/min_distance`, `/kelly/applied_vmax`, `/kelly/applied_inflation`) permettant le suivi et l'enregistrement des données pour analyse ;
- reconfigure dynamiquement les paramètres de Nav2 via les services ROS 2 `set_parameters`, sans nécessiter de redémarrage de la pile de navigation.

Un mode « observation seule » (sans application des paramètres) a été prévu pour valider le calcul du risque avant d'agir sur la navigation.

5.2.3 Adaptation de l'infrastructure existante

L'intégration de cette phase a nécessité deux évolutions de la chaîne mise en place en Phase 1 :

- passage de la localisation sur carte statique (`map_server` + AMCL) à `slam_toolbox` en **mode localisation**, permettant de conserver la carte déjà construite tout en l'étendant automatiquement aux nouvelles zones explorées par le robot ;
- passage du costmap global en mode **fenêtre glissante** (`rolling_window`), pour permettre la planification de trajectoires vers des objectifs situés au-delà des limites de la carte initiale.

6. Résultats et comparaison

6.1 Protocole de comparaison

La comparaison entre navigation classique (Nav2 seul) et navigation risque-adaptative (Nav2 + `kelly_nav`) a été envisagée sur un même scénario de test (entrepôt simulé, mêmes obstacles, même objectif de navigation), selon les indicateurs suivants :

- temps total pour atteindre l’objectif ;
- distance minimale observée aux obstacles au cours de la trajectoire ;
- comportement du robot à l’approche d’un obstacle (ralentissement, augmentation de la distance de sécurité).

6.2 Résultats observés

La navigation risque-adaptative a été testée avec des seuils ajustés ($d_{\text{safe}} = 3.0$ m, $t_{\text{tc_safe}} = 6.0$ s, $kelly_b = 0.6$) permettant d’observer un ralentissement net et une augmentation de la distance de sécurité à l’approche d’obstacles.

Ce comportement a été confirmé par le suivi en direct des topics `/kelly/min_distance` et `/kelly/applied_vmax`. Lors des essais représentatifs, une réduction significative de la vitesse appliquée (passant de 0,5 m/s en régime nominal jusqu’à 0,075 m/s, soit une diminution de 85%) a été mesurée à l’approche d’un obstacle situé à quelques mètres. L’effet est particulièrement visible à l’écran, comme le montre la vidéo de démonstration finale où le robot ralentit nettement à l’approche des étagères de l’entrepôt tout en maintenant une distance de sécurité accrue par rapport à la navigation classique observée en Phase 1.

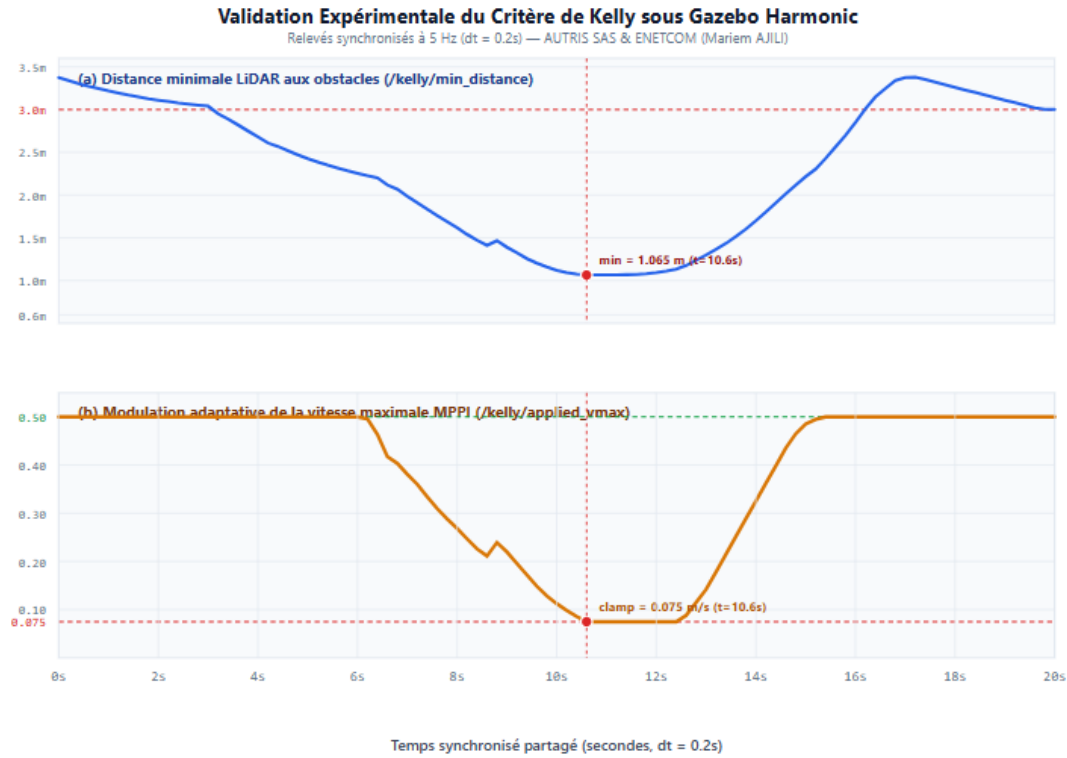


Figure 3 — Validation expérimentale du critère de Kelly sous Gazebo Harmonic (relevés synchronisés à 5 Hz, $dt = 0.2$ s). **Courbe (a)** : distance minimale LiDAR aux obstacles (/kelly/min_distance) — la valeur décroît de 3.3 m jusqu'à un minimum de 1.065 m atteint à $t = 10.6$ s, moment de l'approche maximale de l'obstacle. **Courbe (b)** : modulation adaptative de la vitesse maximale MPPI (/kelly/applied_vmax) — maintenue à 0.5 m/s en régime nominal, la vitesse est progressivement réduite à mesure que le robot s'approche, jusqu'au clamp de 0.075 m/s appliqué à $t = 10.6$ s, puis restaurée à 0.5 m/s lorsque l'obstacle est dépassé. La corrélation temporelle exacte entre les deux courbes illustre le fonctionnement du module *kelly_nav* : plus le risque perçu augmente (distance faible), plus la fraction de Kelly appliquée réduit la vitesse autorisée. Les captures brutes du terminal correspondantes sont disponibles dans le dossier *screenshots/* du dépôt GitHub (Annexe C).

7. Discussion et limites

Le choix de modélisation retenu pour adapter le critère de Kelly à la navigation — bien que cohérent avec la formule originale et avec les objectifs du sujet — reste une interprétation parmi d'autres possibles. D'autres formulations pourraient être envisagées, notamment en intégrant plusieurs sources de risque simultanément (incertitude de localisation, densité d'obstacles, vitesse relative d'obstacles dynamiques) ou en calibrant les paramètres L et G à partir de données réelles plutôt que d'un choix arbitraire.

La limite principale de l'approche réside dans la sensibilité du comportement du robot aux seuils choisis (`d_safe`, `ttc_critical`, `kelly_b`), qui ont dû être ajustés empiriquement pour obtenir un effet visible et exploitable en démonstration, sans validation statistique rigoureuse sur un grand nombre d'essais.

Par ailleurs, l'essentiel du temps du stage (plus de trois semaines) a été consacré à la stabilisation de l'environnement de simulation plutôt qu'au développement algorithmique proprement dit, ce qui a réduit le temps disponible pour explorer des variantes du critère de Kelly ou pour effectuer une évaluation quantitative plus poussée.

8. Conclusion et perspectives

Ce stage a permis de mettre en place une chaîne complète de navigation autonome (SLAM + Nav2) pour un robot mobile simulé, puis d’y intégrer un mécanisme original de gestion adaptative du risque inspiré du critère de Kelly, modulant en temps réel la vitesse et la distance de sécurité du robot en fonction du danger perçu.

Au-delà du résultat technique, ce travail a permis de développer des compétences en robotique (ROS 2, SLAM, Nav2), en simulation (Gazebo), en développement Python, ainsi qu’en résolution de problèmes système, la majeure partie du stage ayant nécessité un travail approfondi de diagnostic et de dépannage d’environnement.

Les perspectives d’amélioration incluent une évaluation quantitative plus poussée (statistiques sur un grand nombre de trajectoires), l’intégration de sources de risque supplémentaires (incertitude de localisation, obstacles dynamiques), ainsi qu’un test sur robot physique pour valider la transférabilité de l’approche au-delà de la simulation.

9. Bibliographie

Critère de Kelly

- Kelly, J. L. Jr. (1956). *A New Interpretation of Information Rate*. Bell System Technical Journal, 35(4), 917-926. <https://doi.org/10.1002/j.1538-7305.1956.tb03809.x>
- MacLean, L. C., Thorp, E. O., & Ziemba, W. T. (2010). *The Kelly Capital Growth Investment Criterion : Theory and Practice*. World Scientific. (ouvrage de référence synthétisant les développements et applications du critère depuis 1956)
- *Kelly Criterion : The Math of Gambling, Proebsting's Paradox, and the Stock Market* [Vidéo]. YouTube. <https://www.youtube.com/watch?v=x9EuFSTnXOE> (support de vulgarisation utilisé en complément pour la préparation de la présentation initiale du sujet)
- Synopsys. *Reinforcement Learning Explained in 90 Seconds* [Vidéo]. YouTube. <https://www.youtube.com/watch?v=C2zw2H1c5Fk> (support de vulgarisation sur les principes de l'apprentissage par renforcement, cité dans le sujet de stage initial)

Kelly et intelligence artificielle / apprentissage par renforcement

- *Reinforcement Learning with Kelly Criterion for Optimal Portfolio Management of AAPL Stock*. Procedia Computer Science, 2025. <https://doi.org/10.1016/j.procs.2025.03.166> (illustre l'intégration du critère de Kelly comme mécanisme de dimensionnement d'action au sein d'un agent d'apprentissage par renforcement — principe transposé

dans ce stage au dimensionnement de la vitesse du robot)

- *The Reinforcement Learning Kelly Strategy*. University of Waterloo (UWSpace). <https://uwspace.uwaterloo.ca/bitstreams/51b3b16e-e397-47bf-9b38-7bf2d55b629e/download>
- *Reinforcement Learning Explained in 90 Seconds* [Vidéo]. Synopsys, YouTube. <https://www.youtube.com/watch?v=C2zw2H1c5Fk> (support de vulgarisation utilisé pour une présentation rapide des principes de l'apprentissage par renforcement)

Navigation robotique sensible au risque (état de l'art)

- Guo, B., Wang, G., Chen, Y., Gao, Y., & Xie, Q. (2025). *Risk-Aware Reinforcement Learning with Dynamic Safety Filter for Collision Risk Mitigation in Mobile Robot Navigation*. Sensors, 25(17), 5488. <https://doi.org/10.3390/s25175488>
- Sun, Z., Diao, X., Wang, Y., Zhu, B.-K., & Wang, J. (2025). *Socially Aware Robot Crowd Navigation via Online Uncertainty-Driven Risk Adaptation*. <https://arxiv.org/abs/2506.14305> (exemples d'approches de navigation intégrant une notion de risque estimé pour moduler le comportement du robot, dans un esprit proche — bien que par des méthodes différentes — de l'approche développée dans ce stage)

Outils et frameworks utilisés

- Open Robotics. *ROS 2 Jazzy Jalisco Documentation*. <https://docs.ros.org/en/jazzy/>
- Macenski, S., Martín, F., White, R., & Ginés Clavero, J. (2020). *The Marathon 2 : A Navigation System*. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). <https://arxiv.org/abs/2003.00368> — voir aussi la documentation officielle : <https://docs.nav2.org/>
- Macenski, S., & Jambrecic, I. (2021). *SLAM Toolbox : SLAM for the dynamic world*. Journal of Open Source Software, 6(61), 2783. <https://doi.org/10.21105/joss.02783>
- dépôt : https://github.com/SteveMacenski/slam_toolbox
- Open Robotics. *Gazebo Harmonic Documentation*. <https://gazebo.org/docs/harmonic>

10. Annexes

- Annexe A — Code source du package `kelly_nav` :
<https://github.com/mariemajili08-star/kelly-nav-slam>
- Annexe B — Liste des nœuds et topics ROS 2 utilisés (relevé réel via `ros2 node list` / `ros2 topic list` pendant les tests) :

Nœuds principaux

- `controller_server`
- `planner_server`
- `bt_navigator`
- `behavior_server`
- `smoother_server`
- `velocity_smoother` (pile Nav2, hébergés dans un conteneur de composants)
- `map_server` (Phase 1, carte statique)
- `sync_slam_toolbox_node` (Phase 2, mode localisation avec extension de carte)
- `amcl` (Phase 1) — remplacé en Phase 2 par la localisation intégrée de `slam_toolbox`
- `local_costmap`
- `global_costmap`
- `kelly_monitor_node` (Phase 2, package `kelly_nav`)
- `rviz2`

Topics principaux

- Perception / état :
 - `/scan`
 - `/odom`
 - `/tf`
 - `/tf_static`
 - `/clock`
- Cartographie et localisation :
 - `/map`
 - `/map_metadata`
 - `/map_updates`

- /pose
- /initialpose
- Navigation :
 - /cmd_vel
 - /goal_pose
 - /plan (via planner_server)
 - /global_costmap/costmap
 - /local_costmap/costmap (et topics associés _raw, _updates, _raw_updates)
- Diagnostic / système :
 - /diagnostics
 - /rosout
 - /parameter_events
 - /bond
- Phase 2 (module kelly_nav) :
 - /kelly/p_safe
 - /kelly/fraction
 - /kelly/ttc
 - /kelly/min_distance
 - /kelly/applied_vmax
 - /kelly/applied_inflation
- SLAM (Phase 2) :
 - /slam_toolbox/feedback
 - /slam_toolbox/graph_visualization
 - /slam_toolbox/scan_visualization
 - /slam_toolbox/update
- Annexe C — Captures d'écran RViz / Gazebo supplémentaires, captures brutes du terminal des topics de diagnostic (/kelly/min_distance, /kelly/applied_vmax) et vidéo de démonstration — voir le dossier `screenshots/` du dépôt GitHub (Annexe A).